Software Product and Process Management (COM435)

Week 5

From Big Stories to Safe Delivery

Khadija Katanani

Software Product and Process Management (COM435)

Part 1: Why splitting matters
Part 2: Splitting techniques
Part 3: Advanced slicing
Part 4: Spikes
Part 5: Integration

Khadija Katanani

Splitting User Stories

moving from writing stories to managing delivery risk

- ❖Why stories fail
- ❖Splitting user stories
- ❖Advanced slicing techniques
- ❖Spike stories
- ❖Integration

We will examine two core problems in Agile:

1. Stories that are too big.
2. Stories that are too uncertain.

Splitting solves the size problem.
Spikes solve the uncertainty problem.

As a user, I want to manage my account.

Let’s analyse this story.

Is this small enough?

Can we estimate it?

Can we deliver it in 1–2 days?

No.

It hides multiple behaviours.

Large stories destabilise sprints.
This is why we split.

Why Split Stories?

- Break into manageable chunks
- Deliver 8–12 stories per sprint
- Separate high value from low value
- Improve planning and productivity

Splitting allows:

- Smaller, testable increments
- Natural backlog prioritisation
- Improved motivation
- Better sprint predictability

Agile thrives on small vertical slices.

INVEST Principles

- Independent
- Negotiable
- Valuable
- Estimated
- Small
- Testable

Splitting must respect INVEST.
We do not break stories randomly.

Each resulting story must:
- Deliver value
- Be estimable
- Be independently testable

Splitting is strategic refinement.

Splitting Pattern #1: Workflow Steps

Example: As a content manager, I can publish a news story to the corporation website. Workflow steps are editorial and legal approval and final review to staging area then to production.

New user stories are:

- **Corporate website:** publish a news story directly to the corporate website.
- **Editorial review:** publish a news story with editor review
- **Legal review:** publish a news story with legal review
- **Staging:** view a news story on a staging site.
- **Production:** publish a news story from the staging site to production.

Splitting Pattern #1: Workflow Steps

Example: As a content manager, I can publish a news story to the corporation website. Workflow steps are editorial and legal approval and final review to staging area then to production.

Many stories hide sequential workflow.
Split along process boundaries.
Deliver earlier usable steps.
This reduces delivery risk.

Splitting Pattern #2: Operations (CRUD)

As a user, I can manage my account.

The word manage covers multiple operations (CRUD).

“Manage account” becomes:

Register: I can register for an account
Login: I can login to my account
Edit: I can edit my account settings
Cancel: I can cancel my account

The word “manage” hides multiple operations.
CRUD abstraction often indicates multiple stories.
Verb generalisation usually signals hidden complexity.

Splitting Pattern #3: Business Rule Variations

As a user, I can search for flights with flexible dates

The word “flexible dates” reveals underlying business rules which would be worthy of their own user stories.

Flexible flight search:

- **n days between x and y:** I can search for flights as “n days between x and y”
- **weekend in December:** I can search for flights as “weekend in December”
- **+/- days:** I can search for flights as “plus/minus days of x and y”

Splitting Pattern #4: Variations in Data

As a content manager, I can create news stories

- …in English
- …in Japanese
- … in Arabic

Splitting Pattern #5: Data entry methods

As a user, I can search for flight between two destinations

User interface for data entry can be simply or more complex

Splitting the user stories:

…using simple date input fields

…with a fancy calendar UI

Splitting Pattern #6: Major Effect

As a user, I can pay for my flight with VISA, MasterCard, American Express

It's clear that the heavy lifting is in the card processing infrastructure. Supporting additional card types is trivial.

Splitting the user stories:

- ...I can pay with one credit card type (VISA, MasterCard, AMEX)
- ...I can pay with all four credit card types (given one is already implemented)

Splitting Pattern #7: Defer Performance

As a user, I can search for flights between two destinations

Make it work → Make it fast

Sometimes performance dominates effort.
Deliver working feature first.
Optimise later.

Splitting the user stories:

- …slow - just get it done and display waiting/searching animation.
- …fast (in under 5 seconds)

Advanced Splitting: Goal Levels

Summary level → User goal level

- Example:
  Manage article →
  - ❖Create
  - ❖View
  - ❖Edit
  - ❖Delete

Goal decomposition prevents oversized stories.

Think:

- •Cloud level (summary)
- •Sea level (user goal)
- •Underwater (subfunction)

Stay at user goal level for delivery.

Split Out NFRs

Separate:

- Performance
- Security
- Scalability
- UX
- Internationalisation

Functional and non-functional concerns can be separated.

This:

- ❖ Improves clarity
- ❖ Enables prioritisation
- ❖ Protects MVP

Non-functional refinements can follow core functionality.

Split by Scenario / Flow

Deliver:

- Happy path first
- Alternative flows later

Deliver the primary happy path first.

Example:
Sign in success before handling:

- Invalid password
- Locked account

This ensures earlier usable value.

Split by Scenario / Flow

Write out steps for core happy path (sign in) to find alternative paths

1. User requests sign in
2. System present sign in form
3. User enters username and password
4. System verifies sign in details for an existing account
5. System verifies account is active
6. System verifies account is not locked
7. System logs in and displays dashboard

Transition: Splitting Reveals Uncertainty

Example:

Investigate credit card processing

Implement credit card processing

Notice something important.
One of these stories is not delivering user value.
It is investigating.

When splitting reveals uncertainty, we introduce a spike.

Spikes - What are they?

- Time-boxed investigation
- Reduces uncertainty
- Does NOT deliver direct user value

A spike is controlled research.
It exists to:

- Reduce risk
- Enable estimation
- Build confidence

It is not open-ended exploration.

Spikes - What are they?

In other words, a spike is a special kind of user story

- Not planned work
- Centred around risk and uncertainty of one or more user stories

- Fact finding stories
- They don’t deliver value directly
- They should remove risk or uncertainty as they are barriers to delivering value

Technical vs Functional Spikes

Technical Spikes:

Technical spikes address architectural uncertainty.

- Evaluate implementation technologies for a particular solution
  Evaluate performance or impact of a new user story
- Build vs buy decisions
- Enhance confidence with a particular technology

Technical vs Functional Spikes

Functional Spikes:

- Address uncertainty about user interaction or UX.
- They are evaluated through
  - ❖Prototyping
  - ❖UI mock ups
  - ❖Hardware prototypes
  - ❖Wire frames
  - ❖Page flows
  - ❖Other user centred techniques

Technical vs Functional Spikes

Both remove blockers.

When Should We Use a Spike?

- Large unknowns
- Estimation impossible
- High technical risk
- New technology

Not every story needs a spike.
Use spikes when:

- Uncertainty prevents commitment.
- Spikes are reserved for critical unknowns.

When Should We Use a Spike?

Use spikes:

- To deliver information needed to confidently identify and estimate the user stories that drive it
- Planning a spike and its associated user stories together in the same iteration is risky
- May be fine for small spikes
- Spike stories should be reserved for critical and larger unknowns

Time Boxing

- Typically ≤ 12 hours
- Added to backlog
- Accepted by Product Owner

Spikes must be time-boxed-> Typically ≤ 12 hours

Without time-boxing:

Research expands indefinitely.
Spikes protect sprint boundaries.

Integration: Splitting vs Spikes

- Splitting manages size.
- Spikes manage uncertainty.
  - Together they:
  - Improve estimation
  - Protect sprint predictability
  - Improve value flow

Example User / Spike Story

User Story

As a consumer, I want to see my daily energy use in a histogram so that I can quickly understand my past, current, and projected energy consumption.

Spike Stories

A technical spike to research how to retrieve past and current energy consumption in order to develop a predictive model.

A functional spike – Prototype a histogram in the web portal and get some user feedback on presentation size, style, and charting.

Closing Reflection

Agile controls:

- Size
- Risk
- Value flow

❖ Agile is not about writing stories. It is about:
Managing complexity,
Controlling risk,
Delivering value incrementally.

Assessment 1– Critical Analysis

Is Splitting Always Beneficial?

Assessment 1– Critical Analysis

Consider a highly interdependent banking system.

- Can splitting introduce coordination overhead?
- When might splitting reduce coherence?
- Does extreme splitting increase integration risk?
- Analyse trade-offs
- Provide example
- Defend position

Assessment 1– Critical Analysis

Is Splitting Always Beneficial?

- Can splitting introduce coordination overhead?

  Splitting is beneficial in Agile because it improves deliverability, estimation accuracy, and value flow. However, it is not universally optimal.

Assessment 1– Critical Analysis

- When might splitting reduce coherence?

  In highly interdependent systems, such as banking or distributed transaction platforms, excessive splitting can introduce coordination overhead. Each small story may require cross-team synchronisation, integration testing, and architectural alignment. This increases communication complexity and may slow delivery rather than accelerate it.

Assessment 1– Critical Analysis

- Does extreme splitting increase integration risk?

  Extreme splitting can also reduce coherence. If stories become too granular, they may no longer represent meaningful user value and instead resemble technical tasks. This violates the “Valuable” and “Independent” elements of INVEST.

Assessment 1– Critical Analysis

- Analyse trade-offs

  Additionally, excessive splitting may increase integration risk. If multiple small slices modify different architectural layers without holistic integration planning, late-stage integration failures may emerge.

Assessment 1– Critical Analysis

- Defend position

Therefore, splitting must be strategic, not mechanical. It should aim for "vertical slices" that deliver perceivable value while preserving architectural integrity.

Assessment 2– Applied Scenario

Split or Spike?

Assessment 2– Applied Scenario

Scenario:

As a user, I want AI to recommend personalised investments.

Questions:

- Should we split this?
- Should we spike?
- Both?
- Justify your reasoning.

This assesses:

- Conceptual clarity
- Decision criteria
- Risk reasoning

Assessment 2– Applied Scenario

Split or Spike?

This story contains both size and uncertainty problems.

From a splitting perspective, it can be decomposed into:

- Basic recommendation engine
- Risk profile collection
- Portfolio visualisation
- Explanation of recommendations

However, there is also significant uncertainty:

- Which AI model to use?
- Regulatory compliance constraints?
- Data privacy requirements?
- Model explainability standards?

Therefore, both splitting and spiking are required.

First, introduce a technical spike to:

- Evaluate AI model feasibility
- Assess regulatory compliance
- Determine data requirements

After reducing uncertainty, split the feature into smaller, testable user stories.

Thus:

Spikes manage uncertainty.

Splitting manages scope.

Both are required sequentially.

Assessment 3– Evaluation of NFR Strategy

Should NFRs Always Be Deferred?

Assessment 3– Evaluation of NFR Strategy

Discuss:

- Using spikes to delay commitment
- Over-spiking
- Research without outcome

How should Scrum Master prevent misuse?

Assessment 3– Evaluation of NFR Strategy

Should NFRs Always Be Deferred?

Deferring non-functional requirements (NFRs) is a common Agile strategy to accelerate MVP delivery. However, deferral is context-dependent.

In systems such as healthcare, aviation, or financial platforms, security, compliance, and reliability are not optional refinements — they are foundational constraints.

For example:
In a banking application, deferring encryption or fraud detection is not acceptable because regulatory requirements demand compliance from the outset.

Therefore:

- Low-risk systems → NFR deferral may be acceptable.
- High-risk or regulated systems → Certain NFRs must be embedded early.

Assessment 4– Spike Misuse

When Do Spikes Become Avoidance?

Assessment 4– Spike Misuse

Discuss:

- Using spikes to delay commitment
- Over-spiking
- Research without outcome

How should Scrum Master prevent misuse?

Assessment 4– Spike Misuse

When Do Spikes Become Avoidance?

Spikes are designed to reduce uncertainty. However, misuse occurs when teams use spikes to avoid commitment or difficult decisions.

Spikes become avoidance when:

- They are repeatedly extended.
- They lack clear learning objectives.
- They produce no actionable outcome.
- They replace architectural decision-making.

Effective spikes must:

- Be time-boxed.
- Have a clear research question.
- Produce measurable findings.
- Inform backlog refinement.

The Scrum Master plays a critical role in preventing spike misuse by:

- Enforcing time limits.
- Demanding clear outcomes.
- Challenging unnecessary research.

Spikes are controlled learning, not indefinite experimentation.

Assessment 5 – Integration Challenge

Large Story Evaluation

As a customer, I want to book international flights with dynamic pricing and multi-currency support.

1. Identify splitting patterns
2. Identify NFR separation
3. Identify spike need
4. Propose time-box

Assessment 5 – Integration Challenge

Step 1 – Splitting Patterns

Using splitting techniques:

Workflow split:

- Search flights
- Select flight
- Payment processing
- Confirmation

Business rule variations:

- Dynamic pricing rules
- Currency conversion handling
- Tax calculation variations

Scenario split:

- Happy path booking
- Payment failure
- Currency mismatch

Step 2 – NFR Separation

Separate:

- Performance (search speed)
- Security (payment encryption)
- Scalability (peak booking periods)

These may be deferred or prioritised depending on system criticality.

Step 3 – Spike Identification

Spikes may be required for:

- Dynamic pricing algorithm feasibility
- Multi-currency conversion integration
- Compliance with international payment regulations

These are high-uncertainty areas.

Assessment 5 – Integration Challenge

Step 4 – Time-Box Proposal

Example:

- 8-hour technical spike to evaluate currency API integration
- 12-hour spike to prototype dynamic pricing model

Spikes should produce:

- Clear feasibility conclusion
- Cost estimate
- Risk assessment

🧠 What a Distinction-Level Answer Demonstrates

- ✅ Identifies both size and uncertainty
- ✅ Uses terminology correctly (INVEST, NFR, vertical slice, spike)
- ✅ Applies concepts to scenario
- ✅ Recognises contextual limitations
- ✅ Balances theory and practice